Universidad de San Carlos de Guatemala

Facultad de Ingeniería

Escuela de Ciencias y Sistemas

Arquitectura de Computadores y Ensambladores 1

Ing. Luis Espino

Aux. Josue Guevara



Proyecto Único – Fase 2
Sistema Inteligente de Evaluación y Predicción Ambiental

Objetivo General:

Desarrollar un sistema IoT capaz de procesar, analizar y predecir datos ambientales, mediante la integración de cálculos estadísticos y modelos predictivos en lenguaje ensamblador ARM64, con almacenamiento en MongoDB y visualización avanzada en Grafana.

Objetivos Específicos:

- Implementar funciones estadísticas básicas en lenguaje ARM64 (media, moda, mediana, varianza, etc.) para analizar datos recolectados en la Fase 1.

- Desarrollar modelos de predicción simples como **Media Móvil** y **Suavizado Exponencial**, codificados en ARM64.
- Establecer un mecanismo de integración entre Python y ARM64, donde el sistema principal (Python) invoque los cálculos escritos en ensamblador con parámetros provenientes de los datos recolectados.
- Analizar y presentar resultados obtenidos de los cálculos estadísticos y modelos predictivos en el dashboard y en la LCD de la Fase 1.
- Fortalecer la comprensión de la arquitectura ARM64 y su uso práctico en escenarios de procesamiento de datos reales.
- Integrar MongoDB Atlas como base de datos para el almacenamiento histórico de sensores y resultados analíticos.
- Configurar y personalizar paneles en Grafana para mostrar datos históricos y resultados analíticos en tiempo real.
- Visualizar en LCD y en el dashboard web los resultados analíticos obtenidos.

Esta fase amplía la funcionalidad de SIEPA para convertirlo en un sistema inteligente completo. Los datos recolectados en la Fase 1 (temperatura, humedad, iluminación, presión) serán almacenados de forma continua en MongoDB y analizados mediante algoritmos estadísticos y modelos de predicción implementados en lenguaje ensamblador ARM64, ejecutados nativamente en la Raspberry Pi.

El sistema Python actuará como orquestador: extraerá subconjuntos de datos de MongoDB, invocará al código ARM64, recibirá los resultados, y los desplegará en:

- La **pantalla LCD local**,
- El **dashboard web** desarrollado en la Fase 1,
- Y **Grafana**, que ofrecerá visualizaciones históricas avanzadas y monitoreo.

Arquitectura del Sistema – Fase 2

Capas del IoT Stack Framework

- **Capa de Percepción**: Sensores conectados a la Raspberry Pi.
- **Capa de Red**: Comunicación MQTT para transmisión de datos.

- **Capa de Procesamiento**: Python + ARM64 para análisis y predicción.
- **Capa de Almacenamiento**: MongoDB Atlas como base de datos central.
- **Capa de Visualización**: LCD, Dashboard Web y Grafana.

Hardware Utilizado

- Raspberry Pi 4 (ARM64).
- Sensores:
    - DHT11/DHT22 (temperatura y humedad).
    - Fotoresistencia (LDR).
    - HC-SR04 o PIR (presencia).
    - BMP280 (presión barométrica).
    - MQ135 (Calidad del aire).
- Actuadores:
    - LEDs, Buzzer, Motor DC.
    - Pantalla LCD I2C.

Flujo General del Sistema

1. **Recolección de Datos**: Python toma lecturas periódicas desde sensores.
2. **Almacenamiento**: Los datos son almacenados automáticamente en MongoDB (cada 30 segundos o cada minuto).
3. **Procesamiento**:
    - Python extrae las últimas *n* muestras (ej. 30, min: 20) desde MongoDB.
    - Se envían al ejecutable ARM64.
    - ARM64 procesa los datos (estadísticas + predicción).
4. **Visualización**:
    - Resultados mostrados en la pantalla LCD.

- Datos calculados en Dashboard Web (Puede ser una ventana extra).
- Datos Guardados de la DB (Puede ser una ventana extra) (min 20 datos).
- Resultados históricos en Grafana (vía conexión a MongoDB).

Estadísticas y Modelos en ARM64

- **Estadísticas**:
  - Media
  - Mediana
  - Moda
  - Mínimo / Máximo
  - Desviación estándar
  - Varianza
- **Modelos de Predicción**:
  - Media Móvil (MA)
  - Suavizado Exponencial (SES)

Integración Python ↔ ARM64

- **Entrada**: Python lee datos desde MongoDB y genera un archivo temporal o pasa argumentos por stdin.
- **Ejecución**: Se invoca el ejecutable ARM64 (compilado en aarch64-linux-gnu).
- **Salida**: ARM64 imprime los resultados en formato legible o archivo.
- **Lectura y Visualización**: Python recoge resultados y los presenta.

MongoDB Atlas

- Almacenamiento centralizado de:
  - Lecturas de sensores.
  - Resultados analíticos.

    - Registros de predicción.
- Colecciones:
    - lecturas_sensores -> {hora-fecha, iluminación, temperatura, etc}
    - estadisticas_resultados -> {hora-fecha, sensorde, media,varianza,etc}
    - predicciones -> {hora-fecha, sensorde, mediaMovil, suavizado}
    - alertas -> {hora-fecha, sensorde, valor}

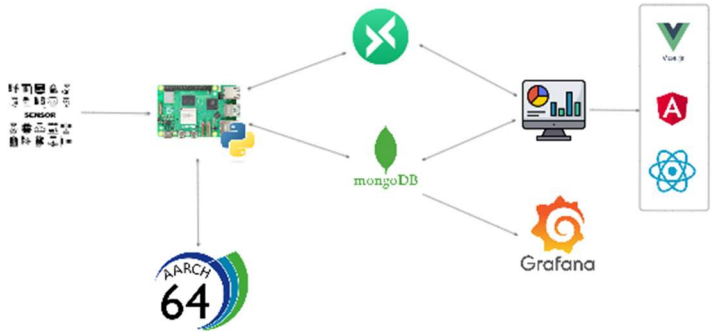
Grafana

- Integrado con MongoDB como fuente de datos (a través de un conector como **Grafana MongoDB Data Source Plugin** o puente intermedio con InfluxDB).
- Paneles:
    - Temperatura histórica.
    - Alertas críticas. (Opcional).
    - Predicciones vs. datos reales.
    - Resumen analítico (media, varianza, etc.).

Visualización LCD

- Pantalla alterna entre los siguientes modos (Cuando se solicite en el dashboard):
    - Valores actuales.
    - Resultados estadísticos.
    - Predicciones.

Diagrama De Fase 2:



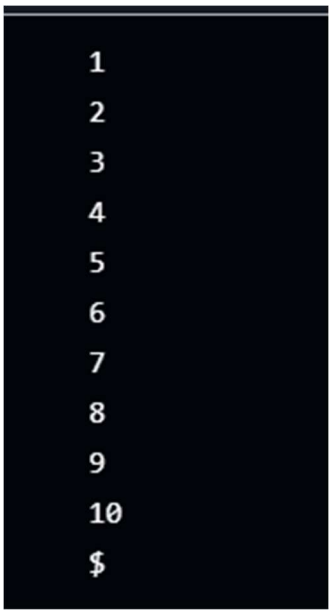
Aplicación TUI en ARM64

Como parte del desarrollo del proyecto, se solicita la implementación de una **aplicación basada en texto (TUI - Text User Interface)** orientada a ejecutarse en arquitectura **ARM de 64 bits**. Esta aplicación debe complementar la funcionalidad actual del proyecto, brindando una interfaz textual que permita al usuario interactuar con las funciones estadísticas y de predicción ya implementadas, a través de un menú numérico.

Requerimientos específicos

1. **Menú de navegación textual**
   El sistema debe presentar al usuario un menú por consola, con opciones numéricas que permitan acceder a las distintas funciones estadísticas y de predicción desarrolladas en el proyecto. La navegación debe ser clara e intuitiva.

2. **Lectura de archivos de datos**
   El programa debe permitir la carga de archivos de texto con datos numéricos o estructurados bajo un formato predefinido. El **formato de los archivos** debe seguirá la siguiente estructura: (Los datos estarán separados por un enter “/n”, y el finalizador del archivo será $)



3. **Filtrado de datos por número de línea**
   Se debe incorporar una opción que permita al usuario seleccionar un subconjunto de los datos cargados, especificando un rango de líneas (por ejemplo, de la línea 100 a la 150). Este filtrado debe aplicarse a las funciones estadísticas y de predicción de forma dinámica.

4. **Limpieza del filtrado**
   Se debe proporcionar una opción para **restaurar la selección completa de los datos**, es decir, eliminar cualquier filtrado aplicado previamente y volver a considerar la totalidad del conjunto de datos.

5. **Exportación de resultados a XML**
   La aplicación debe permitir generar un archivo en formato **XML legible y estructurado según estándares comunes**, conteniendo los resultados de todas las funciones estadísticas y de predicción. Este archivo debe considerar:

   - Todos los datos, si no hay filtrado aplicado.
   - Solo los datos seleccionados, si hay un filtrado activo.

No esta permitido el uso de gcc, el código debe ser ensamblado y enlazado con AS y LD, y ejecutado directamente en la raspberry.

Entregables:

Repositorio de Github:

Se utilizará el **mismo repositorio de la Fase 1**, ahora incluyendo la carpeta:

- Fase2

Contenido requerido:

- Código fuente ARM64 y scripts en Python que lo invocan.
- Manual técnico de la Fase 2.
- **Manual Técnico:**
    - Debe incluir:
        - Descripción del funcionamiento detallado del módulo analítico.
        - Explicación de los cálculos estadísticos y modelos de predicción.
        - Diagrama de flujo de integración Python ↔ ARM64.
        - Consideraciones de arquitectura ARM64 y su rendimiento.
        - Capturas de resultados en LCD, dashboard y Grafana.

Observaciones:

- El ejecutable ARM64 debe ejecutarse **nativamente en Raspberry Pi**.
- Se debe evidenciar claramente la integración Python ↔ ARM64.
- El proyecto debe demostrar **resultados correctos y coherentes** de análisis y predicción con los datos recolectados en la Fase 1.
- Se considerará la optimización y correcta gestión de recursos al implementar los algoritmos en bajo nivel.
- La presentación profesional y claridad técnica del reporte será parte fundamental de la evaluación.